

Building the Portfolio: Hero, Marquee & CTA

A full step-by-step guide — voxel avatar hero, scrolling skill marquee, project showcase, and the closing parallax CTA



↑ The finished hero section: mouse-reveal voxel avatar, animated headline, resume CTA.

Contents

Part I — Hero Section	3
1 Generating the Base Voxel Avatar	3
1.1 Prompt (personal version)	3
1.2 Generalized version (use this if you're not me)	3
2 Generating the Blue-Flame Variant	4
2.1 Unified prompt (single pass)	4
2.2 Reference: the "no flame" base state	4
3 Preparing Transparent PNG Layers	5
4 Coding Agent Prompt — Spotlight / Mask Reveal	5
4.1 Matching CSS to hand off alongside the prompt	6
5 Coding Agent Prompt — Animated Headline (Word Reveal)	7
5.1 Matching CSS keyframes to hand off	7
6 Coding Agent Prompt — CTA Button Animation	7
7 Hero Section — Full Build Order	9
 Part II — Marquee, Featured Projects & CTA	 10
8 Generating the Nether Portal Image	11
8.1 Prompt for Gemini Nano Banana	11
8.2 Removing the Background	11
9 Building the Marquee Section	12
9.1 Vite Project Setup (do this first)	12
9.2 Coding Agent Prompt — Marquee Component	12
9.3 Matching CSS keyframes to hand off	12
10 Enchanted Book & Featured Projects Section	14
10.1 Prompt for the Enchanted Book	14
10.2 Coding Agent Prompt — Featured Projects Section	14
10.3 Removing Your Own Project Links	15
11 Reversed Marquee (Skills Row 2)	16
12 The Llama Image	16
12.1 Prompt for the Llama	16
13 The CTA Section	17
13.1 Prompt for the Night Sky Background	17
13.2 Coding Agent Prompt — CTA Section	18

Part I — Hero Section

1 Generating the Base Voxel Avatar

Start by uploading a real photo of yourself to an image model (Midjourney, GPT image, Gemini, etc.) as the facial reference. The goal is a voxel/blocky portrait that still clearly resembles you — not a generic stylization.

1.1 Prompt (personal version)

Using the uploaded photo as the exact facial reference, match this person's actual face shape and jawline precisely --- do not narrow or slim the jaw or cheeks. Preserve the mustache and goatee exactly as shown, rendered in voxel blocks. Match the hair's actual curl pattern and volume, not straightened or flattened waves. Render in a voxel block art style, black hoodie, sunglasses, hood down, facing the camera directly, centered composition, warm-gray flat background, soft studio lighting.

1.2 Generalized version (use this if you're not me)

Using the uploaded photo as the exact facial reference, recreate this person's real face shape, jawline, and proportions precisely --- do not slim, narrow, or idealize the face. Preserve all real facial hair, hairstyle texture, and hair volume exactly as shown in the photo, rendered in voxel/block form rather than smoothed or simplified. Keep any real accessories (glasses, piercings, etc.) if present. Style: voxel block art, [YOUR OUTFIT --- e.g. black hoodie / jacket], hood down, facing the camera directly, centered composition, plain warm-gray background, soft even studio lighting, no text, no logos.

Swap the [YOUR OUTFIT] bracket for whatever you're actually wearing in the reference photo — the model matches it better than if you invent one.

2 Generating the Blue-Flame Variant

This became the "reveal" layer shown on mouse hover in the hero. Instead of running it in two separate passes (small flame, then intensify), unify it into a single prompt so you get the full effect in one generation.



2.1 Unified prompt (single pass)

Keep this exact image completely unchanged --- same face, same jawline, same facial hair, same sunglasses, same hoodie, same hair, same pose, same warm-gray background, same voxel block style and lighting. Add intense cyan and electric-blue flame/ember effects made of small glowing voxel blocks, rising from the shoulders, wrapping around the neck and collar, and engulfing the top of the head and hair --- not limited to the shoulders. Make the eyes glow bright cyan-white through the sunglasses, as if lit from within like glowing embers behind the lenses. Scatter small floating cube particles in cyan, blue, and orange tones around the head and upper body, matching the voxel block style. Keep the flame in a cool cyan-blue palette (not orange) for a "digital energy" glow rather than literal fire. Keep the same background and overall lighting untouched.

This single prompt combines both original passes (initial flame + "intensify and engulf the head" follow-up) so you don't need a two-step generation — one image, full effect, less back-and-forth with the model.

2.2 Reference: the "no flame" base state



This is the resting/base layer shown before the mouse reveals the flame layer on top of it — used as `transparent-base.png` in the code.

3 Preparing Transparent PNG Layers

For the hover-reveal effect to work, both images need transparent (not gray) backgrounds so they can be stacked and masked in code.

1. Take the base avatar (no flame) and the flame avatar — same pose, same crop.
2. Remove the warm-gray background from both using an AI background remover (remove.bg, Photoshop’s Select Subject, or an image-model prompt like “remove the background, keep only the subject, transparent PNG”).
3. Export both as PNGs with alpha transparency.
4. Name them exactly: `transparent-base.png` (no flame) and `transparent-flame.png` (with flame) — these filenames are referenced directly in the `Hero.tsx` code.
5. Place both in your `/public` folder so they’re accessible at the root path.

4 Coding Agent Prompt — Spotlight / Mask Reveal

This is the core hero effect: the flame image is hidden by default and only revealed in a circular “spotlight” that follows the user’s mouse (or finger, on touch devices), using a CSS mask driven by JS-updated CSS variables.

```
Build a Hero section in React + TypeScript with a mouse-following spotlight reveal effect. There are two stacked full-bleed background images: a base image (transparent-base.png) always visible, and a flame image (transparent-flame.png) stacked on top of it but hidden everywhere except inside a soft circular spotlight that follows the cursor.
Requirements:
- Use a ref on the flame layer div and set two CSS custom properties, --x and --y, to track cursor position.
- Support both mousemove and touchmove/touchstart events (passive listeners) so it works on mobile.
- Smooth the cursor position with linear interpolation (lerp) inside a requestAnimationFrame loop --- do not snap directly to raw mouse coordinates, ease toward them each frame (e.g. smooth += (mouse - smooth) * 0.1).
- Default the spotlight to the center of the viewport on load, before any mouse movement occurs.
- Apply a CSS radial-gradient mask-image to the flame layer using the --x and --y variables as the center, so only a soft-edged circle around the cursor reveals the flame image underneath.
- Clean up all event listeners and cancel the animation frame on unmount.
- Preload both images before rendering the hero content, and show a pulsing skeleton/placeholder until both are loaded, then fade the real content in with a CSS opacity transition.
```

4.1 Matching CSS to hand off alongside the prompt

```
.hero-reveal-img {  
position: absolute;  
inset: 0;  
background-size: cover;  
background-position: center;  
-webkit-mask-image: radial-gradient(  
circle 220px at var(--x, 50%) var(--y, 50%),  
black 0%, black 55%, transparent 100%  
);  
mask-image: radial-gradient(  
circle 220px at var(--x, 50%) var(--y, 50%),  
black 0%, black 55%, transparent 100%  
);  
transition: mask-position 0.05s linear;  
}
```

5 Coding Agent Prompt — Animated Headline (Word Reveal)

The three lines at the top (“Hello there, I’m an AI engineer and a tech content creator.”) animate in word-by-word rather than appearing all at once.

```
Animate a headline so it reveals word-by-word on page load, instead of appearing all at once.
Requirements:
- Take the full headline string and split it into individual words.
- Clear the heading element’s text content, then dynamically create a <span> for each word and append it to the heading.
- Give each span a shared CSS class (e.g. word-reveal) that defines a fade-up keyframe animation (opacity 0 -> 1, translateY 100% -> 0%).
- Stagger the animation by setting an increasing animation-delay per word (e.g. index * 0.05s) so words cascade in left to right instead of all animating at the same time.
- Preserve the space after each word so line-wrapping stays natural.
- Run this as a one-time effect on mount (not on every re-render).
```

5.1 Matching CSS keyframes to hand off

```
.word-reveal {
display: inline-block;
opacity: 0;
transform: translateY(100%);
animation: wordReveal 0.6s ease forwards;
}

@keyframes wordReveal {
to {
opacity: 1;
transform: translateY(0);
}
}
```

6 Coding Agent Prompt — CTA Button Animation

The “View Resume” button has a hover state where a background fill slides in and the arrow circle shifts, giving it a tactile, premium feel.

```
Build a CTA button component with a layered hover animation.
Requirements:
- Structure: an outer link/button wrapper containing three children --- a background fill span, a text span, and a circular icon span with an arrow SVG inside.
- Default state: pill-shaped button, light background, dark text, circular arrow icon on the right in an accent color.
- On hover: the background-fill span scales/slides in from one side to fully cover the button using a CSS transform transition (not a background-color transition, so it animates directionally rather than just fading).
- On hover: the text color inverts to stay readable against the new fill, and the arrow circle nudges slightly (translateX) to suggest forward motion.
- Use CSS transitions with an ease-out timing function around 0.3s--0.4s, so it feels
```

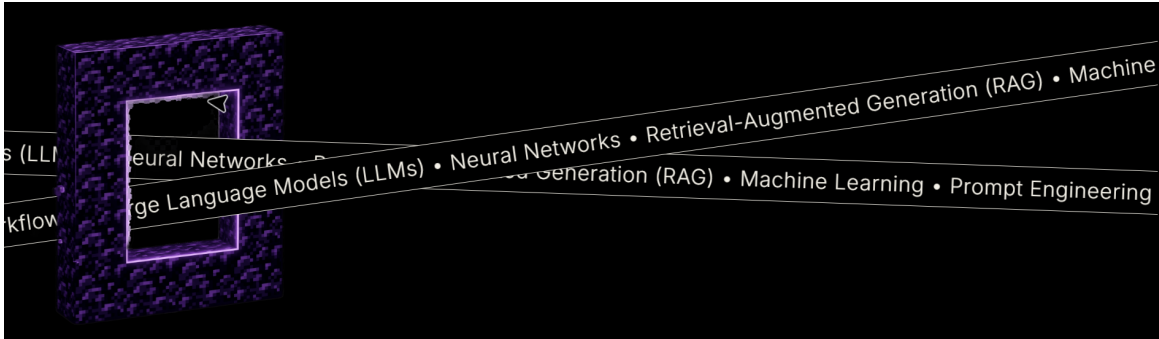
snappy but not instant.

- Keep it accessible: use a real `<a>` or `<button>` element, not a `div` with an `onClick`.

7 Hero Section — Full Build Order

1. **Take/choose a real reference photo** — clear front-facing photo, good lighting, similar pose to what you want in the final avatar.
2. **Generate the base voxel avatar** — use the personal or generalized prompt from Section 1 with your photo as reference.
3. **Generate the blue-flame variant** — use the single unified prompt from Section 2 on the same base image — one pass, not two.
4. **Remove backgrounds from both** — export transparent-base.png and transparent-flame.png with alpha channels, matching filenames exactly.
5. **Drop images into /public** — so they're servable at root paths like /transparent-base.png.
6. **Build the Hero component shell** — layout: big name text, base image layer, flame reveal layer, headline, CTA button, bottom fade.
7. **Hand the coding agent the spotlight prompt** — Section 5 — mouse/touch tracked reveal mask with lerp smoothing and preload skeleton.
8. **Hand the coding agent the headline prompt** — Section 6 — word-by-word staggered reveal animation.
9. **Hand the coding agent the CTA prompt** — Section 7 — layered hover fill + arrow nudge animation.
10. **Test on both desktop (mouse) and mobile (touch)** — confirm the spotlight follows touch events and the skeleton clears once both images preload.

Part II — Marquee, Featured Projects & CTA



↑ The nether-portal-framed scrolling marquee, live on the site.

8 Generating the Nether Portal Image

This is the framing graphic on either side of the marquee text. Generated using Gemini (Nano Banana) — free image generation — then background-removed with Python.

8.1 Prompt for Gemini Nano Banana

```
A Minecraft nether portal frame made of obsidian blocks, rendered in a realistic voxel/blocky 3D style. Rectangular portal shape, glowing purple/violet portal energy swirling inside the frame opening. Obsidian texture with subtle purple undertones and soft ambient glow bleeding onto the surrounding blocks. Camera angled slightly to show depth on one side of the frame. Plain white background, no ground, no other objects, studio product-shot lighting, high detail on the block texture, no text, no logos.
```



8.2 Removing the Background

Give this prompt directly to your coding agent (e.g. Claude Code, Cursor) rather than doing it manually — it will install the library and run the script for you.

```
Write and run a Python script using the rembg library to remove the background from nether-portal.png and save the output as nether-portal-nobg.png with a transparent alpha channel. Install rembg and its dependencies if not already installed, then run the script on the file in the current directory.
```

```
rembg needs onnxruntime under the hood — if your agent hits an install error, have it fall back to pip install rembg[cpu] --break-system-packages and retry.
```


9 Building the Marquee Section

Two horizontal bands of scrolling text (skills/keywords) with the nether portal image anchored at each end, one row scrolling left, the other scrolling right for a crisscross effect.

9.1 Vite Project Setup (do this first)

Since this is a Vite + React + TypeScript app, set the project up before writing any components:

```
npm create vite@latest my-portfolio -- --template react-ts
cd my-portfolio
npm install
npm install framer-motion
npm run dev
```

Place all static assets (nether-portal-nobg.png, enchantedbook.png, llama.png, the sky background, etc.) inside the `/public` folder so they're servable at root paths like `/nether-portal-nobg.png` exactly as referenced in the components below.

9.2 Coding Agent Prompt — Marquee Component

```
Build a reusable Marquee component in React + TypeScript for a Vite app.
Requirements:
- Accept a direction prop ("left" or "right") controlling scroll direction.
- Render a long repeating string of keywords separated by a bullet character,
  duplicated back-to-back inside a wrapping div so the loop is seamless (no visible jump
  when it restarts).
- Animate the text horizontally using a CSS keyframe animation that translates the
  wrapper from 0% to -50% (since the content is duplicated twice, this creates an
  infinite loop). Reverse the animation direction when the direction prop is "right".
- Place a portal-frame image on the left and right edges of the section, both
  absolutely positioned, so the text appears to scroll "through" the frames.
- Add a modifier class (e.g. is--reversed-marquee) that mirrors the frame positions
  for the reversed variant.
- Include a second, independent text row directly below the first, scrolling in the
  opposite direction from the first row, to create a crisscross layered effect.
- Background should be solid black, section should hide any overflow so the duplicated
  text never causes horizontal scrollbars.
```

9.3 Matching CSS keyframes to hand off

```
@keyframes marquee-left {
  from { transform: translateX(0%); }
  to { transform: translateX(-50%); }
}
@keyframes marquee-right {
  from { transform: translateX(-50%); }
  to { transform: translateX(0%); }
}
.marquee--textwrapper-hold {
  display: flex;
  width: max-content;
```

```
animation: marquee-left 22s linear infinite;
}
.marquee--textwrapper-hold.reverse {
animation-name: marquee-right;
}
```

10 Enchanted Book & Featured Projects Section

The heading icon above “Featured Projects” is a glowing enchanted book, generated the same way as the nether portal, then dropped in as a plain heading accent image.

10.1 Prompt for the Enchanted Book

A Minecraft enchanting table with an open enchanted book resting on top, rendered in realistic voxel/blocky 3D style. The book pages glow with a soft magenta-purple enchantment shimmer, faint magical particle sparkles floating around it. Bookshelf base has a warm wood texture with teal/cyan enchanted book spines visible in the side slot. Camera angled slightly from the front-left, isometric-ish perspective. Plain white background, no ground plane, soft studio lighting, high texture detail, no text, no logos.



10.2 Coding Agent Prompt — Featured Projects Section

This mirrors the marquee/parallax patterns already established, applied to a project showcase grid with an expandable “View More” reveal.

Build a Featured Projects section in React + TypeScript for a Vite app using framer-motion.

Requirements:

- A heading row with a small enchanted-book image to the left of the title text as a decorative accent, followed by a short descriptive paragraph, all centered.
- Two floating cloud images positioned absolutely within the section (one upper-right, one lower-left), each with a scroll-linked parallax vertical offset using useScroll + useTransform tied to the section’s scroll progress, moving in opposite directions as the user scrolls.
- A responsive 2-column grid of project cards, showing only the first 2 cards by default.
- Each project card should have a continuous, subtle "breathing" animation (gentle looping translateY and scale oscillation, staggered per card by index) so the grid feels alive even without interaction.
- Each card shows a thumbnail image, title, short description, and two links (demo link and repo link) with small inline icon SVGs.
- A "View More / View Less" pill button below the grid that expands to reveal the remaining project cards with a staggered fade/slide/blur-in entrance animation using AnimatePresence, and collapses them with a matching exit animation.
- When collapsing, smoothly scroll the section back into view instead of letting the sudden height change jump the page.

10.3 Removing Your Own Project Links

Before reusing this component, delete or replace the project data — the array of title/description/demoLink/repoLink objects is personal content, not part of the reusable pattern. Swap in your own projects (or placeholder objects) so the grid renders your work instead of someone else's.

11 Reversed Marquee (Skills Row 2)

Directly beneath the projects grid, a second marquee repeats the pattern from Section 9 but with the direction prop flipped, giving a mirrored scroll direction and reversed frame placement. No new component is needed — reuse the same Marquee component with `direction="right"`.

```
<Marquee direction="right" />
```

12 The Llama Image

Sits on top of the closing CTA card, background removed the same way as the portal and book.

12.1 Prompt for the Llama

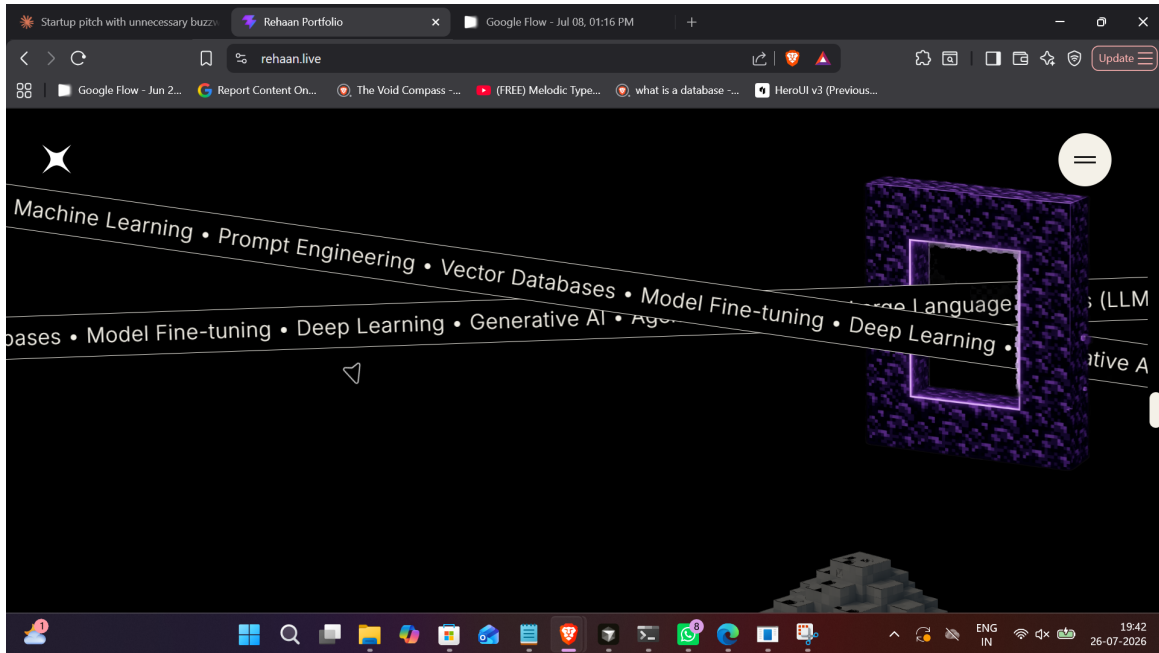
```
A Minecraft llama, cream/white wool coloring, rendered in realistic voxel/blocky 3D style, sitting pose, facing directly toward the camera, calm neutral expression. Soft natural lighting from the front-top, subtle ambient occlusion in the wool texture creases. Plain white background, no ground plane, no other objects, high texture detail, no text, no logos.
```



```
Write and run a Python script using the rembg library to remove the background from llama.png and save the output as llama-nobg.png with a transparent alpha channel. Install rembg if needed, then run it on the file in the current directory.
```

13 The CTA Section

The final section: a parallax night-sky background, the llama perched on top of a glassmorphic card, and contact links.



13.1 Prompt for the Night Sky Background

A Minecraft-style voxel landscape at night, wide panoramic view, snow-dusted mountains on both sides framing a winding river valley through the middle, pine trees scattered along the banks, small distant village silhouettes on the horizon. Dark navy-blue night sky filled with scattered stars, one large glowing full moon with a faint square/pixelated texture visible on its surface, soft blue ambient moonlight washing over the landscape. Rendered in blocky voxel style, wide aspect ratio, no text, no logos, no characters in frame.



13.2 Coding Agent Prompt — CTA Section

Build a closing CTA section in React + TypeScript for a Vite app using `framer-motion` and a `MagicCard` component (spotlight-gradient hover card).

Requirements:

- Full-viewport-height section, black background, content vertically and horizontally centered.
- A large background image (night sky/landscape) absolutely positioned behind everything, oversized (about 180% height) and offset upward, with a scroll-linked parallax vertical transform using `useScroll` + `useTransform` so it drifts slowly as the user scrolls through the section.
- A soft gradient overlay fading from solid black to transparent across the top of the section, layered above the background image, so the image edge blends smoothly into the page above it rather than cutting off hard.
- A centered card container with a mascot image (e.g. an animal) positioned so it appears to sit directly on top of the card, absolutely positioned above the card's top edge, horizontally centered, with a drop shadow for grounding.
- Inside the card: a small uppercase eyebrow label, a bold headline, a short supporting paragraph, and a row of contact links (email, LinkedIn, GitHub, Instagram, etc.) separated by small bullet dots.
- On mobile, wrap the contact links so the last link drops to its own centered row instead of overflowing or wrapping awkwardly mid-row.
- Use a gradient spotlight/hover-glow effect on the card itself (via a `MagicCard`-style component) so the card border/background subtly reacts to mouse position.

If you don't already have a MagicCard component, ask your coding agent to build one: a card wrapper that tracks mouse X/Y relative to itself and renders a radial-gradient glow at that position using a CSS variable, matching the pattern already used in the hero's spotlight-reveal effect.

14 Full Build Order (Marquee, Projects & CTA)

1. **Scaffold the Vite app** — `npm create vite@latest` with the `react-ts` template, install `framer-motion`.
2. **Generate the three decorative assets** — nether portal, enchanted book, and llama, using the prompts in Sections 8, 10, and 12 respectively, via Gemini Nano Banana (or any free image model).
3. **Background-remove all three** — hand each file to your coding agent with the `rembg` prompt; confirm transparent PNGs land in `/public`.
4. **Generate the night-sky background** — Section 13 prompt — and drop it in `/public` too.
5. **Build the Marquee component** — Section 9 prompt — test both `direction="left"` and `direction="right"` render correctly and loop seamlessly.
6. **Build the Featured Projects section** — Section 10 prompt — then swap the placeholder project data for your own.
7. **Drop in the second, reversed Marquee** — Section 11 — directly below the projects grid.
8. **Build the CTA section** — Section 13 prompt — including the `MagicCard` component if you don't have one yet.
9. **Stack all sections in your main App/page file** in order: Hero → Marquee (left) → Featured Projects → Marquee (right) → CTA.
10. **Test scroll performance** — parallax transforms and breathing-card animations can get janky on lower-end devices; ask your agent to add `will-change: transform` to animated layers if you see stutter.

That's the full site build — hero avatar, mask reveal, animated headline, CTA button, scrolling marquee, project showcase, and closing parallax CTA, all with their prompts in one place.